Systèmes d'exploitation avancés

Informatique — Licence 2 (L2)

Cours INF-L2-17

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Plonger au cœur des systèmes d'exploitation : threads et conditions de concurrence, sémaphores, interblocages et leur prévention, mémoire virtuelle et pagination, systèmes de fichiers journalisés, virtualisation et conteneurs, et étude comparée des noyaux Windows et Linux.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Processus et threads : création et synchronisation
• Leçon 2 — Exclusion mutuelle et sémaphores : initiation
• Leçon 3 — Interblocages : détection et prévention
• Leçon 4 — Mémoire virtuelle et pagination
• Leçon 5 — Systèmes de fichiers avancés et journalisation
• Leçon 6 — La virtualisation au niveau du système d'exploitation
• Leçon 7 — Étude comparée : les noyaux de Windows et de Linux
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

En première année, tu as découvert ce que fait un système d'exploitation : lancer des programmes, gérer la mémoire, organiser les fichiers. Ce cours de deuxième année ouvre le capot. Comment plusieurs fils d'exécution peuvent-ils travailler en même temps sans se marcher dessus ? Pourquoi deux programmes peuvent-ils se bloquer mutuellement pour toujours ? Comment un ordinateur fait-il tourner des programmes plus gros que sa mémoire vive ? Ce sont ces mécanismes internes, invisibles pour l'utilisateur, que nous allons démonter un par un.

Chaque leçon s'appuie sur des exemples calculés pas à pas : un compteur partagé dont le résultat devient faux sous tes yeux, un graphe d'allocation où un cycle condamne deux processus, une suite de références de pages dont on compte les défauts en FIFO et en LRU. Ces exemples ne sont pas des illustrations décoratives : ils sont le cœur de l'apprentissage, et savoir les refaire à la main est exactement ce qui est demandé aux examens de licence.

La fin du cours élargit la perspective : systèmes de fichiers capables de survivre à une coupure de courant, machines virtuelles et conteneurs qui ont transformé l'hébergement moderne, et comparaison des deux grandes familles de noyaux, Windows et Linux. À l'issue du cours, tu ne verras plus un système d'exploitation comme une boîte noire, mais comme un ensemble de compromis techniques que tu sais expliquer et justifier.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Distinguer processus et threads, et expliquer le coût relatif de leur création, de leur commutation et de leur communication.
- Identifier une condition de concurrence sur un compteur partagé et la corriger par l'exclusion mutuelle.
- Utiliser les sémaphores (opérations P et V) pour synchroniser un producteur et un consommateur sur un tampon borné.
- Détecter un interblocage dans un graphe d'allocation de ressources et vérifier ses quatre conditions nécessaires.
- Appliquer les stratégies de prévention des interblocages et l'algorithme du banquier sur un état d'allocation simple.
- Traduire une adresse virtuelle en adresse physique et calculer le nombre de défauts de page des algorithmes FIFO et LRU.
- Comparer la virtualisation par hyperviseur et la conteneurisation, puis les architectures des noyaux Windows et Linux.

Leçon 1 — Processus et threads : création et synchronisation

Un processus est un programme en cours d'exécution, doté de son propre espace d'adressage, de ses fichiers ouverts et d'un bloc de contrôle (PCB) où le système range ses registres, son état et sa priorité. Créer un processus coûte cher : il faut allouer la mémoire, initialiser les tables, préparer l'environnement. Changer de processus actif coûte aussi du temps, car il faut sauvegarder tout le contexte de l'un et restaurer celui de l'autre. Cette lourdeur est le prix de l'isolement : deux processus ne peuvent ni lire ni modifier la mémoire l'un de l'autre.

Un thread (fil d'exécution, ou processus léger) est un chemin d'exécution à l'intérieur d'un processus. Tous les threads d'un même processus partagent le code, les variables globales, le tas et les fichiers ouverts ; chacun possède en revanche sa propre pile et ses propres registres. Créer un thread est donc bien plus rapide et léger que créer un processus, et communiquer entre threads se fait simplement en lisant et en écrivant la mémoire partagée. Un navigateur moderne illustre bien l'idée : un thread affiche la page, un autre charge les images, un troisième exécute le JavaScript, et tous manipulent le même document.

Ce partage a un revers dangereux : dès que plusieurs threads modifient les mêmes données en même temps, le résultat peut dépendre de l'ordre exact dans lequel l'ordonnanceur les entrelace, ordre qui change à chaque exécution. On parle de condition de concurrence (race condition) : le programme semble correct, mais il produit de temps en temps un résultat faux, sans message d'erreur. Les exemples classiques sont un compteur incrémenté par deux threads, un solde bancaire débité deux fois, ou une place de concert vendue à deux clients. Toute la suite du cours découle de ce constat : exécuter en parallèle exige de synchroniser.

L'essentiel
Un processus isole sa mémoire mais coûte cher à créer et à commuter ; un thread partage la mémoire de son processus, ce qui le rend léger et rapide, mais expose le programme aux conditions de concurrence sur les données partagées.

Exemple — Compteur partagé : comment deux incrémentations donnent 6 au lieu de 7

```
Situation : un compteur partagé vaut 5.
Deux threads T1 et T2 exécutent chacun : compteur := compteur + 1.
Incrémenter n'est PAS une opération unique ; le processeur fait :
(a) LIRE compteur dans un registre
(b) AJOUTER 1 au registre
(c) ÉCRIRE le registre dans compteur

Entrelacement malheureux choisi par l'ordonnanceur :
t1 T1 LIT compteur -> registre de T1 = 5
t2 T2 LIT compteur -> registre de T2 = 5
t3 T1 AJOUTE 1 -> registre de T1 = 6
t4 T1 ÉCRIT compteur -> compteur = 6
t5 T2 AJOUTE 1 -> registre de T2 = 6 (il avait lu 5)
t6 T2 ÉCRIT compteur -> compteur = 6

Résultat final : 6, alors que deux incrémentations devaient donner 7.
Une mise à jour a été PERDUE : c'est une condition de concurrence.
Conclusion : la suite LIRE-AJOUTER-ÉCRIRE doit être indivisible.
```

À retenir

- Threads d'un même processus : mémoire partagée (code, tas, fichiers), mais pile et registres propres à chacun.
- Une instruction de haut niveau comme « compteur + 1 » se décompose en plusieurs micro-opérations entrelaçables.
- Condition de concurrence = résultat qui dépend de l'entrelacement, donc faux de façon intermittente et difficile à reproduire.

Exercice

Un compte bancaire affiche un solde de 1 000 FCFA. Deux threads de la même application déposent chacun 100 FCFA avec la même méthode en trois micro-étapes (lire le solde, ajouter 100, écrire le solde). Décris un entrelacement qui aboutit à un solde final faux, donne ce solde, et explique en une phrase quelle propriété il faudrait garantir pour éviter le problème.

✔ Corrigé
Entrelacement fautif : T1 lit le solde (1 000) ; T2 lit le solde (1 000) ; T1 ajoute 100 (1 100) et écrit 1 100 ; T2 ajoute 100 à la valeur qu'il avait lue (1 000 + 100 = 1 100) et écrit 1 100. Solde final : 1 100 FCFA au lieu de 1 200 FCFA — un dépôt de 100 FCFA a disparu, exactement comme dans l'exemple du compteur. La propriété à garantir est l'atomicité de la suite lire-modifier-écrire : pendant qu'un thread met à jour le solde, aucun autre thread ne doit pouvoir commencer la même opération ; c'est le rôle de l'exclusion mutuelle étudiée à la leçon suivante.

Leçon 2 — Exclusion mutuelle et sémaphores : initiation

La partie d'un programme qui manipule des données partagées s'appelle une section critique. Pour qu'un mécanisme d'accès soit correct, quatre exigences doivent tenir ensemble : deux threads ne sont jamais simultanément en section critique sur la même ressource (exclusion mutuelle) ; si la section est libre, un thread qui demande à entrer ne doit pas être bloqué par des threads qui, eux, ne demandent rien (progression) ; un thread qui attend doit finir par entrer après un nombre borné de dépassements (attente bornée, sinon c'est la famine) ; et le mécanisme ne doit rien supposer sur les vitesses relatives des threads.

Le sémaphore, proposé par Dijkstra, est un compteur entier protégé par le système, manipulable uniquement par deux opérations indivisibles : P (proberen, essayer) qui décrémente le compteur et bloque l'appelant s'il n'y a plus de jeton disponible, et V (verhogen, augmenter) qui incrémente le compteur et réveille un éventuel thread bloqué. Un sémaphore initialisé à 1 joue le rôle d'un verrou d'exclusion mutuelle : le premier P fait passer la valeur à 0, tout autre thread qui tente P se bloque jusqu'au V du détenteur. Un sémaphore initialisé à N peut, lui, gérer N ressources identiques ou compter des événements.

Le problème du producteur-consommateur est l'application canonique. Un producteur dépose des objets dans un tampon de taille limitée, un consommateur les retire ; il faut empêcher le producteur de déposer dans un tampon plein et le consommateur de retirer dans un tampon vide, tout en protégeant le tampon lui-même. Trois sémaphores suffisent : vide (initialisé à la taille du tampon) compte les places libres, plein (initialisé à 0) compte les objets présents, et mutex (initialisé à 1) protège l'accès au tampon. Le producteur fait P(vide) avant de déposer et V(plein) après ; le consommateur fait l'inverse. L'ordre des P n'est pas indifférent : intervertir P(mutex) et P(vide) peut créer un blocage, comme le montre l'exemple ci-dessous.

L'essentiel
Un sémaphore est un compteur à accès atomique via P (bloquer si rien de disponible) et V (libérer et réveiller). Initialisé à 1, il réalise l'exclusion mutuelle ; des sémaphores compteurs synchronisent producteur et consommateur sur un tampon borné.

Exemple — Producteur-consommateur sur un tampon de 2 cases, tracé pas à pas

```
Tampon de 2 cases. Sémaphores : vide = 2, plein = 0, mutex = 1.
Producteur : P(vide) ; P(mutex) ; déposer ; V(mutex) ; V(plein)
Consommateur : P(plein) ; P(mutex) ; retirer ; V(mutex) ; V(vide)

Étape Action vide plein mutex Tampon
0 état initial 2 0 1 [ . , . ]
1 Prod dépose A 1 1 1 [ A , . ]
2 Prod dépose B 0 2 1 [ A , B ]
3 Prod veut déposer C : 0 2 1 [ A , B ]
P(vide) avec vide = 0 :
le producteur est BLOQUÉ
4 Cons retire A 1 1 1 [ . , B ]
5 Le V(vide) de l'étape 4 0 2 1 [ C , B ]
réveille Prod, qui dépose C

Jamais de dépôt dans un tampon plein, jamais de retrait à vide :
les deux compteurs font respecter les deux limites du tampon.
```

À retenir

- P et V sont atomiques : le système garantit qu'aucun entrelacement ne peut fausser le compteur du sémaphore.
- Sémaphore binaire (0 ou 1) = verrou ; sémaphore compteur = gestion de N ressources ou comptage d'événements.
- Dans producteur-consommateur, on fait toujours P sur le compteur AVANT P(mutex), jamais l'inverse.

Exercice

Un parking de 3 places est modélisé par un sémaphore places initialisé à 3 : une voiture qui entre fait P(places), une voiture qui sort fait V(places). Quatre voitures A, B, C, D tentent d'entrer dans cet ordre, puis B sort, ce qui permet à une voiture en attente d'entrer. Donne la valeur de places après chaque événement et précise quel appel bloque, et quand il est débloqué.

✔ Corrigé
Départ : places = 3. A entre : P fait passer places à 2. B entre : places = 1. C entre : places = 0, le parking est plein. D tente d'entrer : P(places) avec un compteur à 0 bloque D, qui rejoint la file d'attente du sémaphore. B sort : V(places) réveille un attenteur, donc D entre immédiatement et le compteur revient à 0. Au final, A, C et D occupent les trois places et places = 0. Remarque : avec la présentation utilisée dans ce cours, le compteur ne descend jamais sous zéro ; ce sont les processus bloqués dans la file qui matérialisent les demandes en attente.

Leçon 3 — Interblocages : détection et prévention

Un interblocage (deadlock) est une situation où un ensemble de processus s'attendent mutuellement pour toujours : chacun détient une ressource dont un autre a besoin et ne la lâchera qu'après avoir obtenu la sienne. L'image classique est celle de deux véhicules engagés face à face dans une rue trop étroite, chacun exigeant que l'autre

recule. Dans un système, les ressources en jeu sont l'imprimante, un fichier verrouillé, un enregistrement de base de données ou une zone mémoire. Tant qu'aucune intervention extérieure ne casse l'attente, les processus concernés ne progressent plus jamais, et les ressources qu'ils détiennent restent perdues pour tous les autres.

Un interblocage ne peut se produire que si quatre conditions sont réunies simultanément — ce sont les conditions nécessaires de Coffman : (1) l'exclusion mutuelle : chaque ressource n'est utilisable que par un seul processus à la fois ; (2) la détention et attente : un processus détient au moins une ressource tout en attendant d'en acquérir d'autres ; (3) la non-préemption : une ressource ne peut pas être arrachée de force à son détenteur, qui ne la libère que volontairement ; (4) l'attente circulaire : il existe une chaîne fermée de processus P1, P2, …, Pn où chacun attend une ressource détenue par le suivant. Si l'une de ces quatre conditions disparaît, l'interblocage devient structurellement impossible.

Quatre grandes stratégies existent face à ce risque. La prévention casse une des quatre conditions par construction : imposer un ordre global de demande des ressources supprime l'attente circulaire ; exiger toutes les ressources d'un seul coup supprime la détention et attente. L'évitement laisse plus de liberté mais vérifie, avant chaque allocation, que l'état reste sûr — c'est le principe de l'algorithme du banquier, qui refuse toute allocation pouvant mener à une impasse. La détection-récupération laisse l'interblocage arriver, le repère par l'analyse périodique du graphe d'allocation, puis casse le cycle en terminant un processus victime ou en le faisant reculer. Enfin, beaucoup de systèmes réels pratiquent la politique de l'autruche : ignorer le problème, jugé rare, et redémarrer en cas de blocage.

L'essentiel
L'interblocage exige simultanément exclusion mutuelle, détention et attente, non-préemption et attente circulaire. Le traiter revient toujours à supprimer l'une de ces quatre conditions, à éviter les états non sûrs, ou à détecter le cycle et à y mettre fin.

Exemple — Graphe d'allocation : un cycle qui condamne deux processus

```
Ressources à 1 exemplaire : R1 = imprimante, R2 = scanner.
Convention : Pi -> Rj signifie « Pi attend Rj » ;
Rj -> Pi signifie « Rj est détenue par Pi ».

Situation :
P1 détient l'imprimante R1 et demande le scanner R2.
P2 détient le scanner R2 et demande l'imprimante R1.

Arêtes du graphe :
R1 -> P1 (R1 détenue par P1)
P1 -> R2 (P1 attend R2)
R2 -> P2 (R2 détenue par P2)
P2 -> R1 (P2 attend R1)

En enchaînant les arêtes, on obtient le CYCLE :
P1 -> R2 -> P2 -> R1 -> P1

Vérification des 4 conditions de Coffman :
- exclusion mutuelle : imprimante et scanner non partageables.
- détention et attente : P1 garde R1 en attendant R2 ; idem P2.
- non-préemption : personne ne peut forcer une libération.
- attente circulaire : le cycle ci-dessus.
=> INTERBLOCAGE confirmé.
Prévention possible : ordre global — toujours demander R1 avant R2.
```

À retenir

- Quand chaque ressource n'a qu'un seul exemplaire, un cycle dans le graphe d'allocation équivaut à un interblocage.
- Les quatre conditions sont chacune nécessaires : en casser une seule suffit à rendre l'interblocage impossible.
- Prévention = règles permanentes ; évitement = contrôle à chaque allocation ; détection = laisser faire, puis réparer.

Exercice

Cinq philosophes sont assis autour d'une table ronde ; entre chaque paire d'assiettes se trouve une seule fourchette, et il en faut deux pour manger. Chaque philosophe applique la même stratégie : prendre d'abord la fourchette de gauche, puis celle de droite. Montre que si les cinq prennent leur fourchette de gauche exactement au même instant, le système est en interblocage, en reliant la situation aux quatre conditions. Propose ensuite deux corrections différentes.

✔ Corrigé
Si chaque philosophe tient sa fourchette de gauche, chacun détient une ressource (détention) et attend celle de droite, détenue par son voisin (attente) ; une fourchette ne sert qu'à une personne (exclusion mutuelle) et personne ne lâche la sienne (non-préemption) ; la chaîne d'attente forme un cercle complet autour de la table (attente circulaire). Les quatre conditions étant réunies, aucun philosophe ne mangera jamais : c'est un interblocage. Première correction, casser l'attente circulaire : numéroter les fourchettes et imposer de toujours prendre d'abord celle de plus petit numéro — au moins un philosophe devra commencer par sa fourchette de droite, ce qui empêche la formation du cercle. Deuxième correction, casser la détention et attente : ajouter un serveur modélisé par un sémaphore compteur initialisé à 4, qui autorise au plus quatre philosophes à tenter simultanément de prendre leurs fourchettes ; parmi ces quatre, au moins un pourra toujours obtenir ses deux fourchettes, manger, puis libérer sa place.

Leçon 4 — Mémoire virtuelle et pagination

La mémoire virtuelle donne à chaque processus l'illusion de disposer d'un immense espace d'adressage continu, alors que la mémoire vive est limitée et partagée. L'espace virtuel est découpé en pages de taille fixe (souvent 4 Kio), et la mémoire physique en cadres de même taille. Une table des pages, propre à chaque processus, indique pour chaque page le cadre qui la contient, ou bien qu'elle se trouve encore sur le disque. Une adresse virtuelle se lit donc en deux morceaux : le numéro de page, qui sert d'index dans la table, et le déplacement, qui localise l'octet dans la page et reste identique dans l'adresse physique. Le processeur, aidé de la MMU, fait cette traduction à chaque accès mémoire.

Quand un processus référence une page absente de la mémoire vive — son bit de présence est à 0 — le matériel déclenche un défaut de page : le système suspend le processus, va chercher la page sur le disque, la charge dans un cadre libre, met à jour la table, puis relance l'instruction interrompue. Si aucun cadre n'est libre, il faut d'abord évacuer une page victime, choisie par un algorithme de remplacement ; si cette victime a été modifiée, on la réécrit sur le disque. Un défaut de page coûte des millions de fois plus cher qu'un accès mémoire ordinaire : tout l'enjeu est donc de bien choisir les victimes et de garder en mémoire les pages utiles, en exploitant la localité des références.

Trois algorithmes de remplacement structurent le sujet. FIFO évacue la page chargée depuis le plus longtemps, sans regarder si elle sert encore. LRU (Least Recently Used) évacue la page dont la dernière utilisation est la plus ancienne, en pariant que le passé récent prédit le futur proche. OPT, l'algorithme optimal, évacue la page qui sera réutilisée le plus tard dans le futur : impossible à réaliser puisqu'il faudrait connaître l'avenir, mais il sert d'étalon pour mesurer les autres. Surprise célèbre : avec FIFO, ajouter des cadres peut augmenter le nombre de défauts — c'est l'anomalie de Belady, dont LRU est immunisé. Si les défauts deviennent frénétiques, le système s'écroule (thrashing) : il passe plus de temps à échanger des pages qu'à travailler.

L'essentiel
Pagination = pages virtuelles et cadres physiques de même taille, reliés par la table des pages. Un défaut de page déclenche un chargement depuis le disque ; l'algorithme de remplacement (FIFO, LRU, OPT) choisit la victime, et son choix décide des performances.

Exemple — Traduction d'adresse, puis FIFO et LRU sur une suite de références

```
PARTIE 1 — Traduction d'adresse (pages de 1 Kio = 1024 octets)
Adresse virtuelle 2137 :
numéro de page = 2137 // 1024 = 2 ; déplacement = 2137 - 2*1024 = 89.
Si la table des pages indique « page 2 -> cadre 5 », alors :
adresse physique = 5 * 1024 + 89 = 5209.
(Le déplacement ne change jamais : seule la page est traduite.)

PARTIE 2 — Remplacement, 3 cadres, suite : 1 2 3 4 1 2 5 1 2 3 4 5
FIFO (on évacue la page chargée depuis le plus longtemps) :
Réf. : 1 2 3 4 1 2 5 1 2 3 4 5
Cadres : 1 1 1 2 3 4 1 1 1 2 5 5
2 2 3 4 1 2 2 2 5 3 3
3 3 4 1 5 5 5 3 4 4
Défaut?: D D D D D D . . D D . -> 9 défauts
LRU (on évacue la page utilisée il y a le plus longtemps) :
même suite, mêmes cadres -> 10 défauts (détail en TD et TP).
OPT (étalon théorique) -> 7 défauts.
Sur cette suite précise, LRU fait PIRE que FIFO : aucun algorithme
ne gagne sur toutes les suites, mais LRU gagne en moyenne,
grâce à la localité des programmes réels.
Anomalie de Belady (FIFO) : avec 4 cadres, cette même suite donne
10 défauts — PLUS de mémoire, PLUS de défauts !
```

À retenir

- Adresse virtuelle = numéro de page + déplacement ; le déplacement est conservé tel quel dans l'adresse physique.
- Un défaut de page coûte un accès disque : l'objectif est de minimiser leur nombre, pas seulement de « faire rentrer » les pages.
- FIFO peut souffrir de l'anomalie de Belady ; LRU exploite la localité ; OPT est un étalon irréalisable en pratique.

Exercice

Avec 3 cadres de page initialement vides, simule FIFO puis LRU sur la suite de références 1, 2, 3, 1, 4, 2. Donne le nombre de défauts de page de chaque algorithme et le contenu final des cadres.

✔ Corrigé
FIFO : référence 1 (défaut, cadres {1}), 2 (défaut, {1,2}), 3 (défaut, {1,2,3}), 1 (succès), 4 (défaut : on évacue 1, la plus anciennement chargée, cadres {2,3,4}), 2 (succès). Total FIFO : 4 défauts, cadres finaux {2,3,4}. LRU : 1 (défaut, {1}), 2 (défaut, {1,2}), 3 (défaut, {1,2,3}), 1 (succès ; l'ordre d'ancienneté d'utilisation devient 2, 3, 1), 4 (défaut : la page utilisée il y a le plus longtemps est 2, on l'évacue, cadres {3,1,4}), 2 (défaut : la moins récemment utilisée est 3, on l'évacue, cadres {1,4,2}). Total LRU : 5 défauts, cadres finaux {1,4,2}. Ici encore, LRU perd contre FIFO sur cette petite suite construite pour le piéger : en pratique, sur des suites issues de programmes réels, la localité joue en faveur de LRU.

Leçon 5 — Systèmes de fichiers avancés et journalisation

Un système de fichiers moderne fait bien plus que ranger des octets : il maintient des métadonnées (propriétaire, droits, dates, taille), des structures d'index comme les i-nodes d'Unix qui décrivent où se trouvent les blocs d'un fichier, et des mécanismes comme les liens durs (deux noms pour le même i-node) et les liens symboliques (un nom qui pointe vers un autre chemin). L'enjeu critique est la cohérence : à tout instant, les structures sur le disque doivent raconter une histoire sans contradiction — un bloc ne peut pas être à la fois libre et attribué à un fichier, un répertoire ne doit pas pointer vers un i-node inexistant.

Or, une seule opération logique exige plusieurs écritures physiques distinctes. Créer un fichier, par exemple, suppose d'allouer un i-node, l'initialiser, marquer des blocs comme occupés et ajouter une entrée dans le répertoire. Si le courant se coupe au milieu de cette séquence — panne, batterie vide, plantage du noyau — le disque conserve un mélange d'ancien et de nouveau : i-node alloué mais invisible, entrée de répertoire pointant sur du vide, blocs marqués libres alors qu'un fichier les utilise. Sans protection, chaque incident laisse le volume un peu plus corrompu, jusqu'au jour où le système de contrôle (fsck) passe des heures à tenter de réparer les dégâts au démarrage.

La journalisation, utilisée par ext4 sous Linux et par NTFS sous Windows, renverse le problème : avant de modifier les structures définitives, le système écrit dans une zone réservée, le journal, la description complète de l'opération à venir, suivie d'une marque de validation. Ce n'est qu'après cette écriture confirmée qu'il applique réellement les modifications. Après une panne, la reprise est simple et rapide : on rejoue les opérations validées dans le journal et on abandonne celles qui ne le sont pas — le disque revient toujours à un état cohérent, soit entièrement avant, soit entièrement après l'opération. La plupart des systèmes journalisent les métadonnées seules (mode ordered d'ext4), compromis entre sécurité et vitesse ; le mode journal complet, qui copie aussi les données, est plus sûr mais plus lent.

L'essentiel
Une opération de fichier = plusieurs écritures disque ; une panne au milieu laisse le volume incohérent. Le journal enregistre d'abord l'intention complète et validée, puis applique : après un crash, on rejoue ce qui est validé et on oublie le reste.

Exemple — Créer un fichier avec journal : la séquence qui survit à la panne

```
Opération : créer « rapport.txt » dans le répertoire /cours.
Sans journal, l'opération exige 4 écritures séparées :
(1) marquer l'i-node n°57 comme occupé
(2) écrire les métadonnées de l'i-node n°57
(3) marquer le bloc de données n°812 comme occupé
(4) ajouter l'entrée « rapport.txt -> i-node 57 » dans /cours
Panne après (2) : i-node occupé mais invisible => fuite et incohérence.

Avec journalisation (écriture anticipée dans le journal) :
Étape A : écrire dans le JOURNAL le bloc de transaction :
« Début ; anciennes et nouvelles valeurs pour (1)(2)(3)(4) »
Étape B : écrire la marque « TRANSACTION VALIDÉE » dans le journal.
Étape C : appliquer réellement (1)(2)(3)(4) sur le disque.
Étape D : marquer la transaction comme terminée dans le journal.

Panne pendant A ou avant B : transaction ignorée, disque = AVANT.
Panne pendant C ou D : transaction validée -> on la REJOUE au
redémarrage, disque = APRÈS. Dans les deux cas : état cohérent.
```

À retenir

- La cohérence visée est celle des structures du système de fichiers : pas de bloc à la fois libre et utilisé, pas de lien vers le vide.
- Le journal fonctionne comme un brouillon signé : l'opération n'existe que si sa validation est écrite.
- Journaliser seulement les métadonnées (choix courant) protège la structure ; les données en cours d'écriture peuvent encore être perdues.

Exercice

Un utilisateur copie un gros fichier sur une clé formatée sans journalisation. La copie semble terminée, il retire la clé sans l'éjecter proprement et, au branchement suivant, le fichier apparaît avec une taille de 0 octet alors que le nom est bien présent dans le dossier. Explique, à l'aide de la séquence d'écritures, comment cet état a pu se produire, puis ce qu'aurait changé un système journalisé pour la structure du volume.

✔ Corrigé
La création du fichier sur la clé se fait en plusieurs écritures : allocation de l'entrée de répertoire et des métadonnées, puis écriture des blocs de données, puis mise à jour finale de la taille. Le retrait brutal a interrompu la séquence après l'écriture du nom et des métadonnées initiales, mais avant l'enregistrement complet des données et de la taille réelle : le répertoire pointe donc vers un fichier dont la taille enregistrée est restée 0. L'état est « nom présent, contenu absent », une incohérence typique d'opération interrompue. Avec un système journalisé, l'opération aurait d'abord été décrite et validée dans le journal : au branchement suivant, le système aurait soit ignoré la transaction non validée (le fichier n'apparaîtrait pas du tout), soit rejoué la transaction validée (métadonnées complètes et cohérentes). Attention à la limite : la journalisation garantit la cohérence de la structure, pas forcément la survie des données elles-mêmes si elles n'étaient pas encore écrites.

Leçon 6 — La virtualisation au niveau du système d'exploitation

La virtualisation classique repose sur un hyperviseur, couche logicielle qui émule des machines complètes : processeur, mémoire, disque, carte réseau. Chaque machine virtuelle (VM) fait tourner son propre système d'exploitation invité, avec son noyau, totalement isolé des voisines. L'hyperviseur de type 1 s'installe directement sur le matériel (ESXi, Hyper-V en mode serveur, KVM intégré à Linux) ; celui de type 2 s'exécute comme une application sur un système hôte (VirtualBox sur un poste de travail). Le gain est énorme — consolider dix serveurs physiques sous-employés en une seule machine — mais chaque VM embarque un système complet : plusieurs gigaoctets sur disque, un démarrage qui prend des dizaines de secondes, et une double administration.

La virtualisation au niveau du système d'exploitation, popularisée par Docker, prend le contre-pied : au lieu d'émuler du matériel, on partage le noyau du système hôte entre plusieurs environnements isolés appelés conteneurs. Le noyau Linux fournit les deux mécanismes nécessaires : les espaces de noms (namespaces), qui donnent à chaque conteneur sa propre vision des processus, du réseau et des systèmes de fichiers, et les cgroups, qui limitent et comptabilisent son processeur, sa mémoire et ses entrées-sorties. Un conteneur ne contient que l'application et ses bibliothèques : il démarre en une fraction de seconde, pèse souvent quelques dizaines de mégaoctets, et on peut en faire tourner des dizaines sur une machine où tiendraient quelques VM.

Ce partage du noyau est aussi la limite des conteneurs : tous dépendent du même noyau, donc une faille du noyau menace l'ensemble, et un conteneur Linux ne peut pas exécuter un programme Windows sans couche supplémentaire. Les VM offrent une isolation plus forte et la liberté de mélanger les systèmes, au prix de la lourdeur. En pratique, les deux mondes se combinent : dans le cloud, les conteneurs tournent très souvent à l'intérieur de machines virtuelles, l'hébergeur gardant l'isolation des VM entre clients et les développeurs profitant de la légèreté des conteneurs à l'intérieur. Choisir entre les deux, c'est arbitrer entre isolation, densité, vitesse de démarrage et homogénéité des systèmes.

L'essentiel
Hyperviseur = machines virtuelles complètes, chacune avec son noyau : isolation maximale, coût maximal. Conteneurs = un seul noyau partagé, isolé par namespaces et cgroups : démarrage quasi instantané et grande densité, mais dépendance totale au noyau hôte.

Exemple — Deux piles logicielles face à face (schéma en texte)

```
A) VIRTUALISATION PAR HYPERVISEUR
+-------------+ +-------------+ +-------------+
| Application | | Application | | Application |
+-------------+ +-------------+ +-------------+
| OS invité | | OS invité | | OS invité |
| (noyau Win) | | (noyau Lin) | | (noyau Lin) |
+-------------+ +-------------+ +-------------+
| HYPERVISEUR (type 1 ou 2) |
+-----------------------------------------------+
| MATÉRIEL (CPU, RAM, disque, réseau) |
+-----------------------------------------------+
Chaque VM = un ordinateur complet émulé.

B) VIRTUALISATION AU NIVEAU DU SYSTÈME (CONTENEURS)
+-------------+ +-------------+ +-------------+
| Application | | Application | | Application |
| + ses bibl. | | + ses bibl. | | + ses bibl. |
+-------------+ +-------------+ +-------------+
| Moteur de conteneurs (ex. Docker) |
+-----------------------------------------------+
| NOYAU UNIQUE PARTAGÉ (namespaces, cgroups) |
+-----------------------------------------------+
| MATÉRIEL |
+-----------------------------------------------+
Aucun OS invité : les conteneurs partagent le noyau hôte.
```

À retenir

- VM : un noyau par environnement, isolation forte, démarrage lent, image lourde (Go).
- Conteneur : noyau partagé, isolation par namespaces/cgroups, démarrage en moins d'une seconde, image légère (Mo).
- Dans le cloud, on combine souvent les deux : conteneurs dans des VM, isolation entre clients + agilité en interne.

Exercice

Pour chacun des trois besoins suivants, indique s'il faut plutôt une machine virtuelle ou un conteneur, et justifie en une ou deux phrases : (1) faire tourner un ancien logiciel comptable qui n'existe que sous Windows XP, sur un serveur Linux ; (2) déployer rapidement 40 instances identiques d'une petite API web Linux pour absorber un pic de trafic ; (3) héberger sur la même machine les applications de deux clients qui ne doivent absolument rien pouvoir observer l'un de l'autre.

✔ Corrigé
(1) Machine virtuelle : le logiciel exige un autre système (Windows XP) qu'un conteneur partageant le noyau Linux ne peut pas fournir ; la VM émule une machine complète où XP s'installe normalement. (2) Conteneurs : les instances sont identiques, sous Linux comme l'hôte, et doivent démarrer très vite et en grand nombre ; les conteneurs, légers et quasi instantanés, sont faits exactement pour cette montée en charge. (3) Machines virtuelles (éventuellement renforcées) : l'exigence est une isolation maximale entre clients qui ne se font pas confiance ; des noyaux séparés réduisent fortement la surface d'attaque partagée, alors que des conteneurs dépendraient tous du même noyau — une faille du noyau pourrait laisser un client espionner l'autre. On peut aussi combiner : une VM par client, des conteneurs à l'intérieur de chacune.

Leçon 7 — Étude comparée : les noyaux de Windows et de Linux

Le noyau Linux, né en 1991, est un noyau monolithique modulaire : tous les services système (ordonnanceur, mémoire, pilotes, systèmes de fichiers, réseau) s'exécutent dans le même espace mémoire privilégié, ce qui rend les appels entre eux très rapides. Mais une grande partie de ces services, à commencer par les pilotes, peut être compilée sous forme de modules chargeables et déchargeables à chaud, sans recompiler ni redémarrer : le monolithe est découpé en briques amovibles. Son code source est libre et ouvert, développé publiquement par une communauté mondiale ; des milliers de distributions assemblent ce même noyau avec des ensembles logiciels différents.

Le noyau de Windows, dit noyau NT, suit une architecture hybride : il s'inspire des micro-noyaux — qui ne gardent en mode privilégié que le strict minimum et relèguent les services en mode utilisateur — tout en conservant, pour des raisons de performance, de gros sous-systèmes dans l'espace noyau, regroupés sous le nom d'Executive (gestionnaire d'objets, de processus, de mémoire, de sécurité…). Le code source est fermé et développé uniquement par Microsoft. Autour du noyau, Windows centralise la configuration dans une base unique, la base de registre, là où Linux disperse la configuration dans des fichiers texte répartis dans /etc, lisibles et modifiables avec n'importe quel éditeur.

Ces choix d'architecture ont des conséquences concrètes que l'administrateur ressent chaque jour : système de fichiers par défaut (ext4 côté Linux, NTFS côté Windows), modèle de sécurité (utilisateurs et groupes Unix avec permissions rwx, contre des listes de contrôle d'accès très fines sous Windows), philosophie des mises à jour et des pilotes, coût des licences. Mais il ne faut pas caricaturer : les deux noyaux offrent aujourd'hui mémoire virtuelle à pagination, ordonnancement préemptif, multitâche symétrique et journalisation. Ce qui les sépare le plus, au fond, n'est pas la technique pure mais le modèle de développement et de distribution : écosystème ouvert et programmable d'un côté, produit unifié et piloté par un éditeur de l'autre.

L'essentiel
Linux : noyau monolithique modulaire, code libre, configuration par fichiers texte, système de fichiers ext4. Windows NT : noyau hybride, code fermé, configuration centralisée dans le registre, système de fichiers NTFS. Les mécanismes de base (pagination, préemption) sont aujourd'hui très proches.

Exemple — Tableau comparatif des deux noyaux

```
Critère | Linux | Windows (noyau NT)
----------------------+----------------------+----------------------
Architecture | Monolithique | Hybride (micro-noyau
| modulaire | enrichi, Executive)
Code source | Libre et ouvert | Fermé (Microsoft)
Pilotes | Souvent des modules | Pilotes fournis par
| chargeables à chaud | les fabricants
Système de fichiers | ext4 (par défaut) | NTFS (par défaut)
par défaut | |
Configuration | Fichiers texte (/etc)| Base de registre
Mises à jour du noyau | Par la distribution, | Centralisées par
| souvent sans redém. | Microsoft (souvent
| pour les modules | avec redémarrage)
Licence | Gratuite (GPL) | Commerciale
Usage dominant | Serveurs, cloud, | Postes de travail,
| embarqué, supercalc. | entreprises, jeu
Journalisation | Oui (journal ext4) | Oui (journal NTFS)
```

À retenir

- Monolithique ne veut pas dire figé : les modules Linux rendent le noyau adaptable sans recompilation complète.
- Hybride ne veut pas dire micro-noyau pur : Windows NT garde de gros services dans l'espace noyau pour la performance.
- La différence la plus décisive au quotidien est le modèle (ouvert et distribué contre fermé et unifié), plus que la liste des mécanismes.

Exercice

Un institut de formation veut équiper une salle de 30 postes pour enseigner à la fois l'administration serveur et la bureautique, avec un budget logiciel très serré et un technicien qui aime scripter toutes les installations. Propose une répartition argumentée des systèmes sur les postes et/ou en machines virtuelles, en mobilisant au moins quatre critères du tableau comparatif de la leçon.

✔ Corrigé
Proposition cohérente : installer Linux comme système principal sur les 30 postes (licence gratuite — critère coût décisif pour un budget serré ; configuration par fichiers texte et modules, idéale pour enseigner l'administration serveur, usage dominant de Linux ; scripts d'installation et de configuration faciles à automatiser en shell). Ajouter, sur chaque poste ou sur quelques postes seulement, une machine virtuelle Windows (hyperviseur de type 2) pour les cours de bureautique qui exigent les logiciels bureautiques courants et pour montrer l'administration d'un système à registre et à NTFS. Quatre critères du tableau sont mobilisés : licence (gratuite contre commerciale), usage dominant (serveurs contre postes), configuration (fichiers texte scriptables contre registre), et système de fichiers cible des enseignements (ext4 contre NTFS). Une réponse inverse — tout Windows avec des VM Linux — serait défendable si l'institut formait surtout à la bureautique, mais elle coûterait 30 licences et rendrait l'enseignement de l'administration serveur moins naturel.

Travaux dirigés (TD)

TD 1 — Pagination complète : FIFO, LRU et OPT sur une même suite

On dispose de 3 cadres de page, initialement vides, et de la suite de références suivante : 4, 2, 1, 5, 2, 3, 4, 5, 2. 1) Simule l'algorithme FIFO en donnant le contenu des cadres après chaque référence et en signalant chaque défaut de page ; donne le nombre total de défauts et le taux de défauts. 2) Fais de même pour LRU. 3) L'algorithme optimal OPT obtient 6 défauts sur cette suite : explique en deux phrases pourquoi OPT est irréalisable dans un vrai système tout en restant utile au concepteur. 4) Sur cette suite, FIFO bat LRU : cela contredit-il l'affirmation « LRU est meilleur en pratique » ? Justifie.

✔ Corrigé du TD 1

1) FIFO (on évacue la page chargée depuis le plus longtemps) : 4 → défaut, {4} ; 2 → défaut, {4,2} ; 1 → défaut, {4,2,1} ; 5 → défaut, on évacue 4, {2,1,5} ; 2 → succès ; 3 → défaut, on évacue 2, {1,5,3} ; 4 → défaut, on évacue 1, {5,3,4} ; 5 → succès ; 2 → défaut, on évacue 5, {3,4,2}. Total FIFO : 7 défauts, soit un taux de 7/9 ≈ 78 %. 2) LRU (on évacue la page utilisée il y a le plus longtemps) : 4 → défaut, {4} ; 2 → défaut, {4,2} ; 1 → défaut, {4,2,1} ; 5 → défaut, la moins récemment utilisée est 4, {2,1,5} ; 2 → succès (l'ancienneté devient 1, 5, 2) ; 3 → défaut, on évacue 1, {5,2,3} ; 4 → défaut, on évacue 5, {2,3,4} ; 5 → défaut, on évacue 2, {3,4,5} ; 2 → défaut, on évacue 3, {4,5,2}. Total LRU : 8 défauts, taux 8/9 ≈ 89 %. 3) OPT doit connaître les références futures pour choisir la victime dont la prochaine utilisation est la plus lointaine ; un système réel ne connaît que le passé, OPT est donc irréalisable. Il reste utile comme borne inférieure : il dit au concepteur ce que le meilleur algorithme possible obtiendrait (ici 6), donc à quelle distance FIFO ou LRU se situent de l'optimum. 4) Non, pas de contradiction : « meilleur en pratique » est une affirmation statistique sur les suites réelles, qui présentent une forte localité (un programme réutilise les pages récentes). La suite de ce TD est courte et fabriquée pour disperser les références ; elle fait partie des cas où LRU perd. Sur des charges réelles, LRU bat FIFO en moyenne, tout en pouvant perdre sur des suites particulières.

TD 2 — Interblocage : graphe en cycle et algorithme du banquier

Partie A — Trois processus et trois ressources à un exemplaire : P1 détient R1 et attend R2 ; P2 détient R2 et attend R3 ; P3 détient R3 et attend R1. Écris les arêtes du graphe d'allocation, montre le cycle, et conclus. Partie B — Le banquier gère 12 exemplaires d'une même ressource. État actuel : allocations P1 = 2, P2 = 3, P3 = 2 ; besoins maximaux annoncés : P1 = 6, P2 = 5, P3 = 8. (a) Calcule les unités disponibles et les besoins restants. (b) L'état est-il sûr ? Donne une séquence sûre complète. (c) P3 demande 5 unités supplémentaires : le banquier peut-il accepter sans risque ? Justifie par le calcul de l'état résultant.

✔ Corrigé du TD 2
Partie A — Arêtes : R1 -> P1, P1 -> R2, R2 -> P2, P2 -> R3, R3 -> P3, P3 -> R1. En les enchaînant on obtient le cycle P1 -> R2 -> P2 -> R3 -> P3 -> R1 -> P1. Chaque ressource n'ayant qu'un exemplaire, ce cycle est un interblocage : les trois processus sont bloqués définitivement, et les quatre conditions de Coffman sont réunies (ressources exclusives, détention en attendant, pas de préemption possible, attente circulaire). Partie B — (a) Unités allouées : 2 + 3 + 2 = 7 ; disponibles : 12 − 7 = 5. Besoins restants : P1 : 6 − 2 = 4 ; P2 : 5 − 3 = 2 ; P3 : 8 − 2 = 6. (b) État sûr : P2 a besoin de 2 ≤ 5, il peut terminer et libère ses 3 unités → disponibles = 8. P1 a besoin de 4 ≤ 8, il termine et libère 2 → disponibles = 10. P3 a besoin de 6 ≤ 10, il termine. Séquence sûre : P2, P1, P3 — l'état est donc sûr. (c) Si l'on accorde 5 unités à P3 : son allocation passe à 7, les disponibles tombent à 0, et son besoin restant devient 8 − 7 = 1. Les besoins restants sont alors P1 = 4, P2 = 2, P3 = 1, tous strictement supérieurs aux 0 unités disponibles : aucun processus ne peut terminer, l'état est non sûr. Le banquier refuse donc la demande et fait attendre P3, même si physiquement les unités existent : l'évitement préfère un refus temporaire à un risque d'interblocage.

Travaux pratiques (TP)

TP 1 — Programmer un simulateur de remplacement de pages et éprouver l'anomalie de Belady

Ouvre un environnement Python. 1) Écris une fonction fifo(refs, n) qui simule l'algorithme FIFO pour une liste de références refs et n cadres, et qui renvoie le nombre de défauts de page. 2) Écris de même une fonction lru(refs, n). 3) Teste tes deux fonctions sur la suite de la leçon 4 (1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 avec 3 cadres) et sur la suite du TD 1 (4, 2, 1, 5, 2, 3, 4, 5, 2 avec 3 cadres) ; compare aux résultats obtenus à la main. 4) Éprouve l'anomalie de Belady : pour la suite de la leçon 4, calcule fifo(refs, 3) puis fifo(refs, 4) et commente. 5) Bonus : ajoute une fonction horloge (seconde chance) simple ou mesure le temps d'exécution des deux fonctions sur une suite aléatoire de 100 000 références.

✔ Attendus du TP 1
Résultats attendus si les fonctions sont correctes : suite de la leçon 4, 3 cadres → FIFO = 9 défauts, LRU = 10 défauts ; suite du TD 1, 3 cadres → FIFO = 7 défauts, LRU = 8 défauts. Anomalie de Belady : fifo(refs, 3) = 9 mais fifo(refs, 4) = 10 — augmenter le nombre de cadres a augmenté le nombre de défauts, ce qui réfute l'idée que « plus de mémoire donne toujours moins de défauts » pour FIFO (LRU donne 10 défauts avec 3 cadres et 8 avec 4 cadres sur la même suite : aucun effet Belady). Points de vigilance dans le code : en FIFO, il faut conserver l'ordre d'arrivée (une file) et ne pas réordonner lors d'un succès ; en LRU, au contraire, chaque référence — succès ou défaut — doit rafraîchir la position de la page dans l'ordre d'ancienneté, et la victime est la page en tête de cet ordre. Si tes chiffres diffèrent des attendus, l'erreur est presque toujours l'une de ces deux confusions. Pour le bonus, l'algorithme de l'horloge parcourt les cadres en rond en donnant une seconde chance (bit de référence à 1 ramené à 0) aux pages consultées récemment ; ses résultats se situent en général entre FIFO et LRU.

Exercices corrigés supplémentaires

Exercice 1

Algorithme du banquier (un seul type de ressource, 10 unités au total). État actuel : allocations P1 = 3, P2 = 2, P3 = 2 ; maxima annoncés : P1 = 7, P2 = 4, P3 = 9. L'état est-il sûr ? Si oui, donne une séquence sûre ; si non, explique pourquoi aucun ordre ne fonctionne.

✔ Corrigé

Unités disponibles : 10 − (3 + 2 + 2) = 3. Besoins restants : P1 : 7 − 3 = 4 ; P2 : 4 − 2 = 2 ; P3 : 9 − 2 = 7. Cherchons qui peut terminer : P2 a besoin de 2 ≤ 3, il termine et libère ses 2 unités → disponibles = 5. P1 a besoin de 4 ≤ 5, il termine et libère ses 3 unités → disponibles = 8. P3 a besoin de 7 ≤ 8, il termine. L'état est sûr, avec par exemple la séquence P2, P1, P3. Remarque : au départ, ni P1 (4 > 3) ni P3 (7 > 3) ne pouvaient démarrer seuls ; c'est la libération par P2 qui débloque la chaîne, exactement le raisonnement que le banquier automatise avant chaque allocation.

Exercice 2

Un système utilise des adresses virtuelles de 32 bits et des pages de 4 Kio. (a) Combien de pages compte l'espace d'adressage d'un processus ? (b) Si chaque entrée de la table des pages occupe 4 octets, quelle est la taille totale de la table pour un processus qui utiliserait tout son espace ? (c) Traduis l'adresse virtuelle 12345, sachant que la page concernée se trouve dans le cadre 7.

✔ Corrigé

(a) Le numéro de page occupe 32 − 12 = 20 bits (12 bits de déplacement pour 4 Kio = 4096 octets), donc 2^20 = 1 048 576 pages. (b) Table : 1 048 576 × 4 octets = 4 194 304 octets, soit 4 Mio par processus — c'est précisément pour éviter de payer ce coût pour des espaces à peine utilisés que les systèmes réels emploient des tables à plusieurs niveaux. (c) Numéro de page = 12345 // 4096 = 3 ; déplacement = 12345 − 3 × 4096 = 12345 − 12288 = 57. La page 3 étant dans le cadre 7 : adresse physique = 7 × 4096 + 57 = 28672 + 57 = 28729.

Exercice 3

On observe le système suivant : P1 détient R1 ; P2 détient R2 et attend R1 ; P3 attend R2 (P3 ne détient rien). (a) Y a-t-il interblocage ? Justifie en examinant le graphe d'allocation. (b) Donne l'ordre dans lequel les processus peuvent se terminer. (c) P1, avant de se terminer, demande en plus R2. Que se passe-t-il ? Identifie précisément le changement dans le graphe.

✔ Corrigé
(a) Non : les arêtes sont R1 -> P1, R2 -> P2, P2 -> R1 (P2 attend R1) et P3 -> R2 (P3 attend R2). La chaîne d'attente P3 -> R2 -> P2 -> R1 -> P1 ne se referme jamais : il n'y a pas de cycle, donc pas d'attente circulaire, donc pas d'interblocage malgré les attentes en chaîne. (b) P1 ne dépend de personne : il termine et libère R1 ; P2 obtient R1, termine et libère R2 ; P3 obtient enfin R2 et termine. Ordre : P1, P2, P3. (c) La demande de P1 ajoute l'arête P1 -> R2 : le graphe contient alors le cycle P1 -> R2 -> P2 -> R1 -> P1. P1 attend R2 détenue par P2, P2 attend R1 détenue par P1 : les deux se bloquent mutuellement, et P3, qui attend R2, est bloqué par ricochet. Une seule arête a transformé une attente en chaîne saine en interblocage : c'est pourquoi la détection par recherche de cycle est l'outil d'analyse de base.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Ce que les threads d'un même processus partagent, c'est…

   A) Leur pile d'exécution et leurs registres

   B) Le tas, le code et les fichiers ouverts

   C) Rien du tout : ils sont isolés comme des processus

2. Dans l'exemple du compteur de la leçon 1, deux incrémentations donnent 6 au lieu de 7 parce que…

   A) Le processeur ne sait pas additionner

   B) L'ordonnanceur a arrêté un thread définitivement

   C) Les deux threads ont lu la même valeur 5 avant d'écrire : une mise à jour est perdue

3. Un appel P(sémaphore) lorsque le compteur vaut 0…

   A) Provoque une erreur fatale du programme

   B) Bloque l'appelant jusqu'à un prochain V

   C) Fait passer le compteur à −1 et continue

4. Laquelle de ces conditions n'est PAS une des quatre conditions nécessaires de l'interblocage ?

   A) Exclusion mutuelle

   B) Attente circulaire

   C) Présence d'un ordonnanceur préemptif

5. Imposer à tous les processus de demander les ressources dans un ordre global croissant, c'est casser…

   A) L'exclusion mutuelle

   B) L'attente circulaire

   C) La non-préemption

6. Un défaut de page se produit quand…

A) Le disque dur est plein

B) Un processus accède à une page absente de la mémoire vive

C) La table des pages est corrompue

7. Pages de 1 Kio : l'adresse virtuelle 2137 se décompose en…

A) Page 2, déplacement 89

B) Page 21, déplacement 37

C) Page 1, déplacement 1113

8. Sur la suite de la leçon 4 avec 3 cadres, le nombre de défauts vaut…

A) FIFO 10 et LRU 9

B) FIFO 9 et LRU 10

C) FIFO 7 et LRU 7

9. Le rôle premier du journal d'un système de fichiers journalisé est de…

A) Compresser les fichiers pour gagner de la place

B) Accélérer les lectures grâce à un cache

C) Garantir un retour à un état cohérent après une panne

10. La différence fondamentale entre un conteneur et une machine virtuelle est que…

A) Le conteneur partage le noyau du système hôte

B) La machine virtuelle ne peut pas exécuter Linux

C) Le conteneur a besoin d'un hyperviseur de type 1

Corrigés du QCM

1. Réponse B — Code, tas, variables globales et fichiers ouverts sont communs ; seuls la pile et les registres sont propres à chaque thread — c'est ce qui rend les threads légers mais expose aux conditions de concurrence.

2. Réponse C — T1 et T2 lisent tous deux 5, calculent 6 chacun et écrivent 6 : la seconde écriture écrase la première. Le remède est de rendre la suite lire-modifier-écrire indivisible.

3. Réponse B — P bloque le processus ou le thread quand aucun jeton n'est disponible ; il sera réveillé par un V. C'est ce mécanisme qui synchronise, par exemple, le consommateur sur un tampon vide.

4. Réponse C — Les quatre conditions de Coffman sont : exclusion mutuelle, détention et attente, non-préemption et attente circulaire. La préemption de l'ordonnanceur concerne le processeur, pas l'allocation des ressources.

5. Réponse B — Avec un ordre de demande imposé, une chaîne d'attente ne peut jamais se refermer en cycle : l'attente circulaire devient impossible, donc l'interblocage aussi.

6. Réponse B — Le bit de présence de la page vaut 0 : le système charge la page depuis le disque dans un cadre, met à jour la table, puis relance l'instruction.

7. Réponse A — 2137 = 2 × 1024 + 89 : numéro de page 2, déplacement 89. Le déplacement reste inchangé dans l'adresse physique.

8. Réponse B — La simulation donne 9 défauts en FIFO et 10 en LRU (OPT : 7). Cette suite piège LRU, ce qui ne l'empêche pas d'être meilleur en moyenne sur les programmes réels.

9. Réponse C — Le journal enregistre les opérations validées avant leur application : après un crash, on rejoue les validées et on ignore les autres, le volume reste cohérent.

10. Réponse A — Un conteneur n'embarque pas de noyau : il utilise celui de l'hôte, isolé par namespaces et cgroups. La machine virtuelle, elle, fait tourner un système invité complet avec son propre noyau.

Résumé — les notions importantes

• Processus = isolation mémoire mais création et commutation coûteuses ; threads = mémoire partagée, légèreté, mais conditions de concurrence possibles sur les données communes.

• Une opération en apparence simple (compteur + 1) est un enchaînement de micro-étapes entrelaçables : sans protection, une mise à jour peut être perdue.

• Le sémaphore (compteur + opérations atomiques P et V) réalise l'exclusion mutuelle quand il vaut 1 au départ, et synchronise producteur et consommateur via les compteurs vide et plein.

• Interblocage = quatre conditions réunies (exclusion mutuelle, détention et attente, non-préemption, attente circulaire) ; un cycle dans le graphe d'allocation en est la signature quand les ressources sont à un exemplaire.

• Prévention (casser une condition), évitement (algorithme du banquier, états sûrs) et détection-récupération sont les trois réponses organisées au risque d'interblocage.

• Pagination : adresse virtuelle = numéro de page + déplacement conservé ; défaut de page = chargement depuis le disque ; FIFO, LRU et OPT choisissent les victimes, et FIFO peut présenter l'anomalie de Belady.

• Journalisation des systèmes de fichiers : écrire l'intention validée avant d'appliquer garantit la cohérence après une panne ; conteneurs et VM arbitrent entre partage du noyau et isolation complète, comme les noyaux Linux (monolithique modulaire) et Windows NT (hybride) arbitrent les leurs.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Processus et threads : création et synchronisation
Threads d'un même processus : mémoire partagée (code, tas, fichiers), mais pile et registres propres à chacun. ; Une instruction de haut niveau comme « compteur + 1 » se décompose en plusieurs micro-opérations entrelaçables. ; Condition de concurrence = résultat qui dépend de l'entrelacement, donc faux de façon intermittente et difficile à reproduire.

Fiche 2 — Exclusion mutuelle et sémaphores : initiation
P et V sont atomiques : le système garantit qu'aucun entrelacement ne peut fausser le compteur du sémaphore. ; Sémaphore binaire (0 ou 1) = verrou ; sémaphore compteur = gestion de N ressources ou comptage d'événements. ; Dans producteur-consommateur, on fait toujours P sur le compteur AVANT P(mutex), jamais l'inverse.

Fiche 3 — Interblocages : détection et prévention
Quand chaque ressource n'a qu'un seul exemplaire, un cycle dans le graphe d'allocation équivaut à un interblocage. ; Les quatre conditions sont chacune nécessaires : en casser une seule suffit à rendre l'interblocage impossible. ; Prévention = règles permanentes ; évitement = contrôle à chaque allocation ; détection = laisser faire, puis réparer.

Fiche 4 — Mémoire virtuelle et pagination
Adresse virtuelle = numéro de page + déplacement ; le déplacement est conservé tel quel dans l'adresse physique. ; Un défaut de page coûte un accès disque : l'objectif est de minimiser leur nombre, pas seulement de « faire rentrer » les pages. ; FIFO peut souffrir de l'anomalie de Belady ; LRU exploite la localité ; OPT est un étalon irréalisable en pratique.

Fiche 5 — Systèmes de fichiers avancés et journalisation
La cohérence visée est celle des structures du système de fichiers : pas de bloc à la fois libre et utilisé, pas de lien vers le vide. ; Le journal fonctionne comme un brouillon signé : l'opération n'existe que si sa validation est écrite. ; Journaliser seulement les métadonnées (choix courant) protège la structure ; les données en cours d'écriture peuvent encore être perdues.

Fiche 6 — La virtualisation au niveau du système d'exploitation
VM : un noyau par environnement, isolation forte, démarrage lent, image lourde (Go). ; Conteneur : noyau partagé, isolation par namespaces/cgroups, démarrage en moins d'une seconde, image légère (Mo). ; Dans le cloud, on combine souvent les deux : conteneurs dans des VM, isolation entre clients + agilité en interne.

Fiche 7 — Étude comparée : les noyaux de Windows et de Linux
Monolithique ne veut pas dire figé : les modules Linux rendent le noyau adaptable sans recompilation complète. ; Hybride ne veut pas dire micro-noyau pur : Windows NT garde de gros services dans l'espace noyau pour la performance. ; La différence la plus décisive au quotidien est le modèle (ouvert et distribué contre fermé et unifié), plus que la liste des mécanismes.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 4 — Algorithmes de remplacement de pages dans les systèmes d'exploitation: FIFO, LRU, MFU et Opt

Chaîne : B2A Tech · Langue : Français · Niveau : Intermédiaire

Les algorithmes de remplacement de pages expliqués et démontrés : FIFO, LRU, MFU et l'optimal. Parfait pour revoir la leçon 4 en refaisant les simulations en parallèle de la vidéo.

https://www.youtube.com/watch?v=xYCmKxepHOg

Leçon 4 — Comprendre la mémoire paginée, compressée, validée, etc

Chaîne : Ptits trucs & astuces PC · Langue : Français · Niveau : Débutant

Une vulgarisation en français du vocabulaire de la mémoire sous Windows : mémoire paginée, validée, en veille... Utile pour relier les notions de pagination de la leçon 4 à ce que montre le gestionnaire des tâches.

https://www.youtube.com/watch?v=GjtX3gG1KVk

Leçon 6 — Machine Virtuelle vs Conteneur Docker : Comprendre la Différence

Chaîne : Fabrice Chrzanowski – Passionate IT Training · Langue : Français · Niveau : Intermédiaire

Hyperviseur et système invité d'un côté, conteneurs et noyau partagé de l'autre : la vidéo compare fonctionnement, avantages et cas d'usage des deux approches de la leçon 6, avec des exemples concrets d'entreprise.

https://www.youtube.com/watch?v=ctwvADb9aek

Sources et références

- Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.
- Silberschatz, Galvin & Gagne — « Operating System Concepts » (Wiley) : le manuel de référence sur les processus, la synchronisation, les interblocages et la mémoire virtuelle.
- Andrew S. Tanenbaum & Herbert Bos — « Systèmes d'exploitation » (Pearson) : une présentation claire des mécanismes internes, de la pagination aux systèmes de fichiers.
- William Stallings — « Operating Systems: Internals and Design Principles » (Pearson) : pour approfondir threads, sémaphores et virtualisation.
- Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !